Cours 13 – Coder un réseau neuronal

Des premières équations à un prototype complet de restauration d'image

Projet IPR – Documentation Scholar

31 octobre 2026

Aucun prérequis en apprentissage automatique. Exemples originaux exécutables.

Table des matières

1 Ce que nous allons construire **4**

1.1 Trois significations de « partir de zéro » 4

1.2 Vocabulaire initial 4

2 Préparer un atelier Python isolé **6**

2.1 Environnement virtuel 6

2.2 Exemples fournis 7

3 Les mathématiques indispensables, sans raccourci **8**

3.1 Scalaires, vecteurs, matrices et tenseurs 8

3.2 Fonction, pente et dérivée 8

3.3 Moyenne, variance et probabilité 9

4 Premier apprentissage : retrouver une droite **10**

4.1 Un pas calculé à la main 11

5 Du neurone au réseau multicouche **12**

5.1 Pourquoi XOR demande de la non-linéarité 12

6 Comprendre et programmer la rétropropagation **15**

6.1 La règle de la chaîne 15

6.2 Un cas où les formules se simplifient 15

6.3 Ce que PyTorch automatise 16

7 Choisir une perte et un optimiseur **17**

7.1 L'erreur choisie définit ce que le réseau préfère 17

7.2 SGD, momentum et Adam 17

7.3 Initialisation et stabilité 18

8 Construire les données avant de grossir le modèle **19**

8.1 Trois groupes et deux niveaux de décision 19

8.2 D'où vient une cible propre ? 19

9 Des nombres aux images : la convolution **21**

9.1 Dimensions et contexte 21

10 Concevoir un petit U-Net complet **23**

10.1 Pourquoi prédire un résidu 24

11 Coder et entraîner ce U-Net avec PyTorch **25**

11.1 Lire le code sans magie 26

11.2 Ce que ce laboratoire prouve et ne prouve pas 27

12 Cas IPR : couleur, bruit et valeurs négatives **28**

12.1 Modèle physique et conditionnement 28

12.2 Ce que nous ne déléguerons pas silencieusement 29

13 Passer du laboratoire à un projet complet **30**

13.1 RAM, VRAM et temps 31

14 Tester, diagnostiquer et décider **32**

14.1 Ordre conseillé 32

14.2 Mesures et validation 33

15 Parcours pratique et exercices corrigés **34**

15.1 Parcours en cinq ateliers 34

15.2 Questions et solutions 34

16 Références et suite dans IPR **36**

Chapitre 1

Ce que nous allons construire

Un réseau neuronal est un programme paramétrable. Ses paramètres numériques, appelés poids et biais, sont ajustés à partir d'exemples. Il ne comprend pas automatiquement la photographie et ne découvre pas une vérité physique sans données appropriées. Il apprend des relations qui peuvent être utiles, mais aussi des biais, des défauts et des raccourcis présents dans les exemples.

Notre objectif est de comprendre chaque étape : représenter les données, calculer une prédiction, mesurer l'erreur, calculer ses dérivées, modifier les paramètres, puis vérifier que le résultat fonctionne sur des exemples nouveaux. Nous commencerons avec quelques nombres, puis passerons aux images et à un U-Net.

1.1 Trois significations de « partir de zéro »

On peut écrire les calculs sans bibliothèque, programmer une architecture connue avec PyTorch, ou entraîner ses propres poids sans modèle préentraîné. Ces trois choix sont différents. Nous ferons les deux premiers, puis un petit ajustement synthétique pour comprendre le troisième. Un entraînement photographique sérieux reste un projet de données et de qualification distinct.

Une architecture éprouvée n'impose pas de reprendre les poids de ses auteurs. Inversement, posséder ses poids n'assure ni leur qualité ni des droits suffisants sur les images ayant servi à les apprendre. Séparer architecture, implémentation, poids, données et moteur d'exécution évite beaucoup de confusions.

1.2 Vocabulaire initial

Terme	Sens concret
Entrée	Nombres fournis au programme, par exemple une image bruitée.
Cible	Résultat souhaité pour un exemple, par exemple une référence moins bruitée.
Prédiction	Résultat actuellement calculé par le réseau.
Paramètre	Nombre modifiable pendant l'apprentissage : poids ou biais.
Hyperparamètre	Choix de conception : largeur, taux d'apprentissage, taille des crops.
Perte	Nombre qui mesure une erreur selon une règle choisie.
Gradient	Dérivées de la perte par rapport aux paramètres.
Entraînement	Calcul répété des prédictions, gradients et mises à jour.
Inférence	Utilisation des poids figés pour calculer une sortie.
Généralisation	Capacité à fonctionner sur des exemples non utilisés pour les réglages.

Chapitre 2

Préparer un atelier Python isolé

Une variable nomme une valeur, une liste regroupe des valeurs, une boucle répète une opération et une fonction rassemble un calcul réutilisable. Python permet d'expérimenter ces notions sans compiler un exécutable. Les fichiers du cours restent indépendants du moteur de production d'IPR.

```
def prediction(x, weight, bias):
    return weight * x + bias

values = [-1.0, 0.0, 1.0]
for x in values:
    print(prediction(x, 2.0, 1.0))
```

2.1 Environnement virtuel

Un environnement virtuel contient ses propres paquets Python. Il évite de modifier les dépendances d'un autre agent ou celles des campagnes alpha. Depuis un dossier de travail dédié, les commandes suivantes illustrent une installation CPU ; adapter le nom de l'interpréteur installé sur la machine.

```
python -m venv .venv-cours
.venv-cours\Scripts\python.exe -m pip install numpy==2.3.5
.venv-cours\Scripts\python.exe -m pip install torch==2.9.1+cpu --index-url https://
    download.pytorch.org/whl/cpu
```

Sous Linux/macOS, l'interpréteur de cet environnement se trouve généralement dans `.venv-cours/bin/python`. Les versions ci-dessus correspondent au témoin CPU de ce cours sur Windows/Python 3.14, pas à une qualification universelle. Pour un GPU, choisir explicitement un paquet compatible avec le pilote et le système : avoir une carte NVIDIA ne suffit pas à rendre CUDA utilisable.

2.2 Exemples fournis

Depuis la racine du worktree documentaire :

```
python docs/scholar/references/reseaux-exemples.py
python docs/scholar/references/reseaux-exemples.py --torch
```

La première commande utilise seulement la bibliothèque standard. La seconde nécessite PyTorch dans l'interpréteur choisi. Aucun exemple ne télécharge de poids, ne lit de photo, ne crée de fichier ou ne modifie un RAW. Le petit exemple PyTorch est limité à deux threads CPU et à une image artificielle.

Chapitre 3

Les mathématiques indispensables, sans raccourci

3.1 Scalaires, vecteurs, matrices et tenseurs

Un scalaire est un nombre, comme une température. Un vecteur est une suite de nombres : un pixel RGB peut être noté $x = (R, G, B)$. Une matrice est un tableau à deux axes. Un tenseur généralise ces tableaux à plusieurs axes. Le mot ne signifie pas à lui seul qu'il s'agit d'un réseau ou d'une opération compliquée.

Une image RGB de hauteur H et largeur W contient $3HW$ valeurs. En mémoire intercalée, l'ordre peut être $H \times W \times 3$. Les couches de convolution PyTorch utilisent couramment $N \times C \times H \times W$, où N est le nombre d'images traitées ensemble et C le nombre de canaux. Changer l'ordre des axes demande une permutation, pas un simple changement de dimensions.

Le produit scalaire $w^T x = \sum_i w_i x_i$ combine plusieurs entrées. Une couche dense calcule $z = Wx + b$. Si x a d composantes et z en a m, W a $m \times d$ coefficients et b en a m. Il y a donc $md + m$ paramètres.

3.2 Fonction, pente et dérivée

Une fonction associe une sortie à une entrée. La dérivée décrit comment la sortie change pour une petite variation de l'entrée. Pour $f(w) = w^2$, $f'(w) = 2w$. Une fonction avec plusieurs paramètres possède une dérivée par paramètre : leur ensemble est le gradient.

La différence finie permet de vérifier une dérivée programmée :

$$f'(w) \approx \frac{f(w+\varepsilon) - f(w-\varepsilon)}{2\varepsilon}.$$

Un ε trop grand introduit une erreur d'approximation ; trop petit, il expose aux arrondis. Ce contrôle sert à déboguer un petit calcul, pas à entraîner efficacement des millions de poids.

3.3 Moyenne, variance et probabilité

La moyenne estime un niveau central. La variance mesure une dispersion : $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2]$. L'écart-type est sa racine carrée. Une loi gaussienne est décrite par une moyenne et une variance, mais le bruit d'un capteur réel n'est pas toujours gaussien, indépendant ou uniforme sur toute l'image.

Une probabilité prédit une fréquence ou une incertitude sous des hypothèses. Un débruiteur ne doit pas être confondu avec une mesure certaine de la valeur réelle d'un pixel. Plusieurs images propres peuvent expliquer une observation très bruitée ; aucun programme ne recrée une information détruite avec garantie.

Chapitre 4

Premier apprentissage : retrouver une droite

Supposons que des exemples suivent $y = 2x + 1$, mais que le programme ne connaisse pas les coefficients. Nous choisissons $\hat{y} = wx + b$, initialisons $w = b = 0$, puis mesurons l'erreur quadratique moyenne :

$$L = \frac{1}{n}\sum_{i=1}^{n}(wx_i + b - y_i)^2.$$

En posant $e_i = wx_i + b - y_i$, on obtient

$$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_i e_i x_i, \qquad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_i e_i.$$

La descente de gradient effectue $w \leftarrow w - \eta \partial L/\partial w$ et $b \leftarrow b - \eta \partial L/\partial b$. Le taux η règle la longueur du pas. Le signe moins cherche à descendre la perte, sans garantir de trouver son meilleur minimum pour tout problème et tout taux.

```
def linear_demo():
    samples = [(-1., -1.), (0., 1.), (1., 3.), (2., 5.)]
    w, b = 0., 0.
    for _ in range(1000):
        dw = db = 0.
        for x, target in samples:
            error = w*x+b-target
            dw += 2*error*x/len(samples)
            db += 2*error/len(samples)
        w -= .05*dw
        b -= .05*db
    assert abs(w-2) < 1e-6 and abs(b-1) < 1e-6
    print(f'Linear: w={w:.8f}, b={b:.8f}')
```

Ce programme retrouve environ $w = 2$ et $b = 1$. Il contient déjà l'essentiel d'une boucle d'apprentissage : données, prédiction, erreur, gradients, mise à jour. Une architecture plus grande répète ces principes avec davantage de paramètres.

4.1 Un pas calculé à la main

Pour $x = 2$, $y = 5$, $w = 2$, $b = 2$, la prédiction vaut 6. Avec $L = \frac{1}{2}(\hat{y} - y)^2$, la perte vaut 0,5, les gradients valent 2 pour w et 1 pour b. À $\eta = 0,1$, les nouveaux paramètres sont 1,8 et 1,9. La prédiction devient 5,5 et la perte 0,125. Le facteur $1/2$ explique l'absence de facteur 2 dans cette variante ; toujours lire la définition exacte de la perte.

Chapitre 5

Du neurone au réseau multicouche

Un neurone calcule d'abord $z = w^T x + b$, puis une activation $a = \phi(z)$. Sans activation non linéaire, plusieurs couches affines successives restent une seule transformation affine : empiler des couches ne suffit donc pas.

Activation	Formule	Utilité et limite
ReLU	$\max(0, z)$	Simple ; dérivée nulle côté négatif.
Sigmoïde	$1/(1 + e^{-z})$	Sortie entre 0 et 1 ; saturation aux extrêmes.
Tanh	$\tanh(z)$	Sortie signée bornée ; peut saturer.
SiLU	$z/(1 + e^{-z})$	Activation lisse, utilisée dans le TP.
Identité	z	Sortie sans borne imposée, utile au RGB signé.

Une activation interne ReLU ne force pas nécessairement une sortie RGB positive : la dernière couche peut combiner les caractéristiques avec des poids signés. En revanche, une sigmoïde finale contraindrait réellement la sortie à $(0, 1)$, ce qui ne correspond pas au signal étendu d'IPR.

5.1 Pourquoi XOR demande de la non-linéarité

XOR vaut 1 si deux bits diffèrent, sinon 0. Une seule frontière droite ne sépare pas ses quatre exemples. Une couche cachée peut construire plusieurs régions, puis les combiner. Le TP entraîne huit neurones cachés avec tanh et une sortie sigmoïde ; les gradients sont calculés à la main, sans PyTorch.

```
def sigmoid(z):
    # Avoid overflow on either tail.
    if z >= 0:
        return 1/(1+math.exp(-z))
    ez = math.exp(z)
    return ez/(1+ez)


def xor_demo():
    rng = random.Random(17)
    data = [([0., 0.], 0.), ([0., 1.], 1.), ([1., 0.], 1.), ([1., 1.], 0.)]
    count = 8
    weights = [[rng.uniform(-1, 1) for _ in range(2)] for _ in range(count)]
    biases = [rng.uniform(-.2, .2) for _ in range(count)]
    output_weights = [rng.uniform(-1, 1) for _ in range(count)]
    output_bias = 0.
    def forward(x):
        hidden = [math.tanh(sum(a*b for a,b in zip(row,x))+bias)
                  for row,bias in zip(weights,biases)]
        logit = sum(a*b for a,b in zip(output_weights,hidden))+output_bias
        return hidden, sigmoid(logit)
    for _ in range(3000):
        gw = [[0.,0.] for _ in range(count)]
        gb = [0.]*count
        gv = [0.]*count
        gc = 0.
        for x,target in data:
            hidden,prediction = forward(x)
            delta = (prediction-target)/len(data)  # BCE + sigmoid derivative
            gc += delta
            for j in range(count):
                gv[j] += delta*hidden[j]
                dh = delta*output_weights[j]*(1-hidden[j]**2)
                gb[j] += dh
                for k in range(2):
                    gw[j][k] += dh*x[k]
        for j in range(count):
            biases[j] -= .5*gb[j]
            output_weights[j] -= .5*gv[j]
            for k in range(2):
                weights[j][k] -= .5*gw[j][k]
        output_bias -= .5*gc
    predictions = [forward(x)[1] for x,target in data]
    assert all((p > .8 if y else p < .2) for p,(_,y) in zip(predictions,data))
    print('XOR:',[round(p,5) for p in predictions])
```

Ce petit problème n'est pas une photographie miniature : il sert à comprendre les paramètres et la circulation des dérivées. Le nombre de neurones d'un modèle image ne peut pas être choisi en extrapolant les quatre exemples XOR.

Chapitre 6

Comprendre et programmer la rétropropagation

6.1 La règle de la chaîne

Si $a = \phi(z)$ et $z = wx + b$, alors $\partial L/\partial w = (\partial L/\partial a)\phi'(z)x$. Chaque opération multiplie le gradient reçu par sa propre dérivée locale. Un graphe de calcul garde les dépendances pour appliquer cette règle dans l'ordre inverse du calcul direct.

Pour une couche dense, en notant $\delta = \partial L/\partial z$:

$$\frac{\partial L}{\partial W} = \delta x^T, \qquad \frac{\partial L}{\partial b} = \delta, \qquad \frac{\partial L}{\partial x} = W^T \delta.$$

Pour un lot, sommer ou moyenner les contributions selon la réduction de la perte. Ne pas moyenner deux fois par erreur : cela change la taille des gradients.

6.2 Un cas où les formules se simplifient

La perte binaire est $L = -[y \log(p) + (1-y)\log(1-p)]$. Avec $p = \text{sigmoid}(z)$, $\partial L/\partial z = p - y$. Pour tanh, la dérivée vaut $1 - a^2$. Ce sont les deux relations utilisées dans le TP XOR. Les produits utilisent les poids du même état avant leur mise à jour ; les modifier au milieu de la rétropropagation mélangerait deux versions du réseau.

6.3 Ce que PyTorch automatise

Un tenseur avec `requires_grad=True` participe au suivi des gradients. `loss.backward()` remplit les champs `.grad`. L'optimiseur les utilise ensuite ; il ne calcule pas lui-même la vérité ou la fonction de perte. Les gradients s'accumulent par défaut, d'où `zero_grad()` à chaque pas, sauf accumulation de lots explicitement voulue et normalisée.

```
optimizer.zero_grad(set_to_none=True)
prediction = model(inputs)
loss = (prediction - target).square().mean()
loss.backward()
optimizer.step()
```

`model.eval()` change certains modules comme dropout ou BatchNorm, mais ne désactive pas les gradients. `torch.no_grad()` désactive leur suivi, mais ne met pas automatiquement le modèle en mode évaluation. Pour une inférence habituelle, employer les deux, avec des contrats d'entrée vérifiés.

Chapitre 7

Choisir une perte et un optimiseur

7.1 L'erreur choisie définit ce que le réseau préfère

La MSE pénalise fortement les grands écarts et conduit théoriquement à la moyenne conditionnelle si le modèle et l'optimisation sont idéaux. Une ambiguïté sur les détails peut donc produire du lissage. La MAE conduit à une médiane conditionnelle ; elle n'assure pas davantage la préservation de chaque texture.

Une perte de Charbonnier moyenne $\sqrt{e^2 + \varepsilon^2}$ et fournit un compromis lisse. Des termes de gradients ou de couleur peuvent compléter la perte pixel, mais leurs coefficients changent l'objectif : les qualifier par ablation plutôt que les additionner parce qu'ils semblent utiles.

Pour un débruiteur scientifique, une perte perceptuelle ou adversariale ne doit pas devenir un moyen silencieux d'inventer de la matière. Une image agréable peut être moins fidèle. Choisir d'abord fidélité, biais photométrique et détails connus, puis une revue visuelle sur de nouvelles scènes.

7.2 SGD, momentum et Adam

SGD applique une mise à jour à partir d'un lot. Le momentum conserve une moyenne des gradients pour stabiliser la direction. Adam suit deux moyennes :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Après corrections de biais, $\theta_{t+1} = \theta_t - \eta\hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$. Le carré et la division sont composante par composante. Adam n'empêche pas le surapprentissage ; AdamW traite la décroissance des poids séparément de l'adaptation du pas. Comparer un petit nombre de choix explicitement définis.

7.3 Initialisation et stabilité

Initialiser tous les neurones cachés de façon identique maintient des symétries qui limitent leur apprentissage. Les initialisations aléatoires contrôlées, comme Xavier ou He selon les activations, évitent ce piège. Fixer la graine permet de répéter un essai, sans garantir une identité numérique entre GPU.

Le TP initialise seulement la dernière couche résiduelle à zéro : la sortie initiale vaut exactement l'entrée, tandis que le corps est aléatoire. Au premier pas, le gradient apprend surtout cette tête ; une fois ses poids non nuls, les couches précédentes reçoivent leur gradient. Ce n'est pas l'initialisation de tout le réseau à zéro.

Chapitre 8

Construire les données avant de grossir le modèle

Un dataset associe les entrées aux cibles et métadonnées. Un chargeur compose les lots. Un crop est une région de l'image, pas une nouvelle scène indépendante. Les transformations géométriques doivent être identiques pour l'entrée et sa cible. Un décalage d'un pixel peut enseigner du flou au lieu du débruitage.

8.1 Trois groupes et deux niveaux de décision

Le jeu d'entraînement met à jour les poids. La validation aide à choisir les hyperparamètres et le checkpoint. Le test final évalue le choix figé ; il ne sert pas à régler une nouvelle variante. Si l'on consulte sans cesse le même test pour choisir, il devient indirectement une validation.

Séparer par scènes, séances et, pour tester une généralisation inter-boîtiers, par modèles de capteurs. Mettre deux images d'une rafale dans deux groupes ne crée pas cette indépendance. Conserver identités, empreintes, provenance, autorisations et raisons d'exclusion dans une mémoire versionnée.

8.2 D'où vient une cible propre ?

Une moyenne de prises alignées atténue un bruit temporel indépendant ; elle conserve le bruit fixe commun et peut lisser le mouvement. Une photo basse ISO en pose longue est un candidat, pas une vérité identique par définition : éclairage, saturation, exposition et signal capteur doivent être raccordés. Une image Lightroom débruitée est le résultat d'un autre algorithme, non une mesure certaine de la scène.

Le bruit synthétique permet de connaître la cible, mais un bruit gaussien ajouté après dématriçage ne reproduit pas tout un capteur. Mélanger données réelles qualifiées et simulation documentée est une stratégie à étudier, sans confondre leurs niveaux de preuve.

Chapitre 9

Des nombres aux images : la convolution

Une couche dense appliquée à tous les pixels aurait énormément de paramètres et ne partagerait pas naturellement les motifs spatiaux. Une convolution utilise un petit ensemble de poids qui glisse sur l'image. Le même détecteur local peut ainsi répondre à un bord situé en plusieurs endroits.

Dans les bibliothèques, l'opération dite convolution est généralement une corrélation croisée, sans retournement du noyau :

$$Y_{o,h,w} = b_o + \sum_{c,u,v} K_{o,c,u,v} X_{c,h+u,w+v}.$$

Les conventions d'origine, padding et stride complètent cette formule. Avec un noyau $k \times k$, C_{in} canaux et C_{out} sorties, le nombre de paramètres est $k^2 C_{in} C_{out} + C_{out}$ si le biais est activé.

Pour 3 entrées, 8 sorties et $k = 3$, on obtient 224 paramètres. Pour les six canaux du TP, la première couche $6 \to 8$ en compte 440. Une carte de caractéristiques n'est plus un canal rouge ou bleu : c'est une représentation apprise, dont la signification n'est pas imposée individuellement.

9.1 Dimensions et contexte

Avec padding p, dilatation d et stride s, une dimension devient

$$H_{out} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} + 1 \right\rfloor.$$

Deux convolutions 3×3, stride 1, voient un voisinage théorique 5×5. Des réductions de résolution augmentent plus vite le contexte, mais peuvent perdre des détails. Le contexte théorique ne signifie pas que tous les pixels ont la même influence effective.

Le padding nul invente des valeurs en bordure ; réflexion et réplication ont leurs propres hypothèses. Le tuilage doit conserver un halo cohérent. Dès que le réseau emploie des statistiques globales, un halo fini ne garantit plus la même sortie qu'une image entière.

Chapitre 10

Concevoir un petit U-Net complet

U-Net associe un encodeur qui réduit la résolution à un décodeur qui la remonte. Les connexions latérales apportent des caractéristiques fines de l'encodeur au décodeur. Le principe vient d'un travail de segmentation ; son adaptation à la restauration demande un objectif et une sortie appropriés.

Étape	Dimensions pour une entrée 32×32	Rôle
Entrée	$1 \times 6 \times 32 \times 32$	RGB et sigma par canal
Encodeur1	$1 \times 8 \times 32 \times 32$	Caractéristiques fines
Encodeur2	$1 \times 16 \times 16 \times 16$	Contexte intermédiaire
Centre	$1 \times 32 \times 8 \times 8$	Contexte réduit
Décodeur2	$1 \times 16 \times 16 \times 16$	Fusion avec encodeur2
Décodeur1	$1 \times 8 \times 32 \times 32$	Fusion avec encodeur1
Tête	$1 \times 3 \times 32 \times 32$	Résidu signé

La concaténation additionne les nombres de canaux :32+16=48 au premier décodeur, puis16+8=24 au second. Elle n'est pas une addition pixel à pixel. Le rééchantillonnage remonte les dimensions mais ne recrée pas à lui seul l'information perdue ; les convolutions et connexions latérales complètent le calcul.

10.1 Pourquoi prédire un résidu

On peut prédire l'image propre directement ou une correction r_θ : $\hat{x} = y - r_\theta(y, \sigma)$. Le résidu n'est pas automatiquement une réalisation exacte du bruit physique. La formulation facilite une identité initiale et une interface de force, mais ne garantit ni absence de biais ni protection de chaque détail.

La largeur 8 et les deux réductions du TP donnent 29 995 paramètres. C'est un petit témoin pédagogique et technique, pas une taille minimale démontrée pour les photos Canon. Il ne faut pas confondre le nombre de couches, le nombre de canaux, le nombre de poids et le nombre de pixels de contexte.

Chapitre 11

Coder et entraîner ce U-Net avec PyTorch

Voici le laboratoire complet : imports locaux, définition des blocs, classe, données synthétiques, optimiseur, calcul direct et rétropropagation. La fonction peut être exécutée avec l'option `-torch` du script fourni.

```
def torch_demo():
    try:
        import torch
        from torch import nn
        from torch.nn import functional as F
    except ImportError as error:
        raise SystemExit('PyTorch absent: use an isolated environment or omit --torch')
    from error
    torch.set_num_threads(2)
    torch.set_num_interop_threads(1)
    torch.manual_seed(17)
    torch.use_deterministic_algorithms(True)
    def block(inputs,outputs):
        return nn.Sequential(nn.Conv2d(inputs,outputs,3,padding=1),nn.SiLU(),
                             nn.Conv2d(outputs,outputs,3,padding=1),nn.SiLU())
    class TinyUNet(nn.Module):
        def __init__(self):
            super().__init__()
            self.e1=block(6,8)
            self.e2=block(8,16)
            self.middle=block(16,32)
            self.d2=block(48,16)
            self.d1=block(24,8)
            self.head=nn.Conv2d(8,3,1)
            nn.init.zeros_(self.head.weight)
```

```
            nn.init.zeros_(self.head.bias)
        def forward(self,image,sigma):
            a=self.e1(torch.cat((image,sigma),dim=1))
            b=self.e2(F.avg_pool2d(a,2))
            c=self.middle(F.avg_pool2d(b,2))
            d=self.d2(torch.cat((F.interpolate(c,size=b.shape[2:],mode='nearest'),b),1))
            e=self.d1(torch.cat((F.interpolate(d,size=a.shape[2:],mode='nearest'),a),1))
            return image-self.head(e)
    model=TinyUNet()
    axis=torch.linspace(-.1,1.2,32)
    clean=axis[None,None,None,:].expand(1,3,32,32).clone()
    sigma=torch.full_like(clean,.05)
    noisy=clean+sigma*torch.randn_like(clean)
    original=noisy.clone()
    assert torch.equal(model(noisy,sigma),noisy)
    optimizer=torch.optim.Adam(model.parameters(),lr=.002)
    initial=((noisy-clean)**2).mean().item()
    model.train()
    for _ in range(32):
        optimizer.zero_grad(set_to_none=True)
        loss=(model(noisy,sigma)-clean).square().mean()
        loss.backward()
        assert all(p.grad is None or torch.isfinite(p.grad).all() for p in model.
    parameters())
        optimizer.step()
    model.eval()
    with torch.no_grad():
        restored=model(noisy,sigma)
        final=(restored-clean).square().mean().item()
    assert torch.equal(original,noisy)
    assert torch.isfinite(restored).all() and restored.shape==clean.shape
    assert final < initial
    print('U-Net CPU:',sum(p.numel() for p in model.parameters()),'parameters')
    print(f'Single-patch MSE: {initial:.8f} -> {final:.8f}')
    print('Teaching-only memorization check, NOT photographic validation; no weights
    saved.')
```

11.1 Lire le code sans magie

Une sous-classe de `nn.Module` enregistre ses couches et leurs paramètres. `forward` décrit le calcul. Appeler `model(image,sigma)` passe par ce calcul et les mécanismes du framework. `model.parameters()` fournit les poids à l'optimiseur ; les tenseurs de l'image ne deviennent pas des paramètres parce qu'ils traversent le réseau.

`torch.cat` réunit RGB et cartes sigma. Le pooling réduit la taille. `interpolate` rejoint les dimensions exactes des connexions latérales, y compris lorsque certaines dimensions sont impaires. Les couches finales ne comportent ni sigmoïde ni clamp. Le modèle travaille en flottants signés.

11.2 Ce que ce laboratoire prouve et ne prouve pas

Une perte qui baisse sur une seule rampe prouve au mieux que des gradients et une mise à jour fonctionnent dans ce cas. Elle ne montre pas de généralisation, de fidélité des cheveux ou de performance sur une nuit lunaire. Un modèle peut mémoriser une image et échouer sur toutes les autres.

Pour une vraie boucle, remplacer l'image fixe par des lots du dataset, garder un jeu de validation indépendant, calculer les métriques sous `no_grad`, et sélectionner le checkpoint par la règle fixée avant le test final. Les augmentations, normalisations et masques font partie du modèle au sens opérationnel : les conserver avec les poids.

Chapitre 12

Cas IPR : couleur, bruit et valeurs négatives

Un réseau ne sait pas spontanément si un triplet représente du sRGB encodé, du Rec.2020 linéaire ou des axes caméra. Le même nombre n'a pas le même sens dans ces domaines. Définir ordre RGB/BGR, gains, transfert, blanc, noir, unités, orientation et stade du pipeline est aussi important que choisir les couches.

La cible étudiée pour IPR est un RGB caméra déjà balancé, après RCD/médiane, avant optique et DCP lorsque la source pré-optique est disponible. Les valeurs négatives sous le noir sont des observations utiles, pas nécessairement des erreurs de fichier. Des valeurs au-dessus de 1 peuvent également être finies et recevables dans un domaine étendu.

12.1 Modèle physique et conditionnement

Dans une approximation capteur, pour un signal attendu $s \geq 0$, $\mathrm{Var}(n \mid s) \approx as + b$. Le terme de signal provient notamment du comptage des photons ; le terme constant représente une partie du bruit de lecture. Après gains, matrices et dématriçage, corrélations et variance changent : trois cartes sigma ne décrivent pas toute la covariance.

Donner une carte de bruit au réseau peut rendre ses décisions plus explicites, mais cette carte doit être calibrée ou son incertitude représentée. L'ISO seul n'est pas une puissance de bruit universelle. Changer de boîtier peut changer bruit fixe, réponse spectrale, niveaux et traitements intégrés.

12.2 Ce que nous ne déléguerons pas silencieusement

Les conversions couleur connues et les profils restent des transformations déterministes. Le réseau propose une correction, pas un nouveau choix créatif de couleur. Mesurer séparément biais dans les noirs, conservation des gris, écarts des couleurs saturées et détail. L'absence de clamp final autorise les valeurs signées, mais ne suffit pas à apprendre leur bon traitement.

La force zéro contourne exactement le modèle. Une interpolation $x_t = y + t(\hat{x} - y)$ donne une commande simple, pas une garantie de qualité monotone ; tester les valeurs intermédiaires. Ne pas débruiter cumulativement la sortie précédente à chaque mouvement du curseur : repartir de la source immuable et de la recette courante.

Chapitre 13

Passer du laboratoire à un projet complet

Partie	Responsabilité
Contrat	Axes, unités, domaine couleur, valeurs valides et erreurs.
Données	Identités, partitions, références, masques et droits.
Modèle	Architecture, configuration, initialisation et poids.
Entraînement	Lots, perte, optimiseur, état de reprise et graines.
Évaluation	Métriques, scènes indépendantes, contrôles et revue visuelle.
Inférence	Conversion des buffers, tuilage, ressources et annulation.
Déploiement	Runtime, licences, empreintes et compatibilité des opérateurs.
Suivi	Versions, résultats JSON, limites et décisions de validation.

Un checkpoint de reprise contient généralement poids, état de l'optimiseur, compteur d'étapes, configuration et états aléatoires. Une livraison d'inférence n'a pas besoin de tout cet état. Ne pas exécuter un pickle ou un script inconnu pour obtenir des poids ; provenance, chargement limité aux tenseurs et empreintes doivent être contrôlés.

Exporter vers ONNX ne garantit pas une identité numérique ou la prise en charge de chaque opérateur. Comparer les sorties de référence et exportées sur les mêmes entrées, y compris tailles impaires, négatifs, constantes et bords. Le passage FP32 vers FP16 ou une quantification int8 demande une qualification propre ; des économies mémoire ne justifient pas une dérive cachée.

13.1 RAM, VRAM et temps

Les poids ne sont qu'une partie de la mémoire. Ajouter activations, gradients, états de l'optimiseur, lots et buffers de transfert. Adam conserve notamment des états par paramètre. L'entraînement coûte donc plus de mémoire que l'inférence du même réseau. Une image RGB float32 de 32 Mpx occupe 384 Mo décimaux avant le moindre réseau.

Commencer avec de petits crops et un seul lot, mesurer les pics, puis augmenter progressivement. Le disque peut conserver les RAW et un cache borné, mais ne remplace pas directement la VRAM d'une opération. Fixer un budget commun aux tâches pour éviter plusieurs réseaux qui pensent chacun disposer de toute la machine.

Chapitre 14

Tester, diagnostiquer et décider

14.1 Ordre conseillé

Vérifier d'abord dimensions et unités. Faire ensuite fonctionner un petit lot, contrôler les gradients puis surajuster volontairement quelques exemples pour détecter une erreur de raccordement. Passer enfin à des données indépendantes avec un protocole figé. Un échec dès les petites étapes évite un entraînement coûteux dont les résultats seraient ininterprétables.

Symptôme	Hypothèses à vérifier
Perte NaN	Valeur invalide, division/logarithme, pas trop grand, débordement.
Perte constante	Gradient coupé, poids non donnés à l'optimiseur, tête non reliée.
Belle image très lisse	Cible floue, perte favorisant la moyenne, bruit mal simulé.
Couleur modifiée	RGB/BGR, transfert, gains, normalisation ou cible incohérents.
Bon entraînement, mauvais test	Fuite de données, surapprentissage, domaine non couvert.
Coutures	Padding, halo, fusion ou statistiques spatiales du réseau.
RAM croissante	Graphes conservés, sorties accumulées, chargeurs ou caches non bornés.

14.2 Mesures et validation

Avec une référence admissible, MSE et PSNR sont utiles ; $\mathrm{PSNR} = 10 \log_{10}(M^2 / \mathrm{MSE})$, où M doit être défini. Pour un domaine signé étendu, annoncer $M = 1$ par convention ne signifie pas que toutes les valeurs sont bornées à1. SSIM et mesures perceptuelles ajoutent des indications, mais aucune moyenne unique ne couvre la fidélité.

Sans référence propre, une baisse de l'énergie haute fréquence peut être du lissage de texture. Employer des masques fixes, des détails connus, des comparaisons visuelles à taille native et des mesures de biais/couleur. Une rafale lunaire sur trépied reste affectée par dérive et turbulence : ne pas appeler « vérité » son empilement sans qualification.

Chronométrer chargement froid et calcul chaud séparément ; synchroniser le GPU avant de lire le chronomètre, sinon on mesure parfois seulement une mise en file. Inclure les transferts et le raccord des tuiles dans la latence utilisateur. Annulation et résultat obsolète doivent empêcher toute publication partielle.

Chapitre 15

Parcours pratique et exercices corrigés

15.1 Parcours en cinq ateliers

Atelier 1 : exécuter la régression, changer un point et expliquer les gradients. Atelier 2 : vérifier une dérivée par différences finies puis lire le code XOR. Atelier 3 : tracer toutes les dimensions du U-Net et compter ses paramètres. Atelier 4 : exécuter le TP PyTorch, changer une seule hypothèse et conserver le témoin initial. Atelier 5 : préparer un dataset avec provenance et scènes réservées avant de lancer une expérience photographique.

15.2 Questions et solutions

1. Combien de paramètres dans une couche dense $4 \to 3$? $4 \times 3 + 3 = 15$, avec un biais par sortie.

2. Et une convolution 3×3, $8 \to 16$? $9 \times 8 \times 16 + 16 = 1168$. Cela ne dépend pas de la largeur de l'image, contrairement au coût de calcul et à la mémoire d'activations.

3. Pourquoi une sigmoïde finale gêne-t-elle le RGB signé ? Elle impose des sorties entre 0 et 1. Une observation négative ne peut plus être représentée, même si l'architecture et les poids étaient parfaits.

4. Pourquoi force zéro ne veut-elle pas dire sigma zéro ? Un réseau à sigma nul peut encore modifier l'image. Une identité exacte se garantit par le contrat de contournement, pas par un espoir sur les poids.

5. Peut-on partager les crops d'une photo entre train et test ? Pas pour mesurer une généralisation à de nouvelles scènes : textures, éclairage et traitements restent communs. Garder la scène entière dans un groupe.

6. Pourquoi la variance d'une moyenne ne baisse-t-elle pas toujours en $1/N$? Les covariances ajoutent des termes : $\mathrm{Var}(N^{-1}\sum X_i) = N^{-2}\sum_{ij}\mathrm{Cov}(X_i, X_j)$. Le bruit commun ne disparaît pas avec une simple moyenne.

7. Pourquoi appeler `eval()` et `no_grad()` ? Le premier règle le comportement de certains modules ; le second désactive le suivi des gradients. Ils n'ont pas la même responsabilité.

8. Le TP apprend-il à débruiter nos RAW ? Non. Il ajuste une rampe artificielle avec bruit gaussien et vérifie le câblage. Il n'a ni dataset Canon, ni référence photographique, ni validation indépendante.

9. Une baisse du bruit apparent suffit-elle ? Non : vérifier matière, fins contours, biais, couleur et hallucinations, sinon un simple flou peut gagner le mauvais concours.

10. Pourquoi ne pas commencer par un modèle énorme ? Sans données et contrats fiables, sa capacité accélère aussi la mémorisation des biais ; coût et complexité rendent le diagnostic plus difficile.

Chapitre 16

Références et suite dans IPR

Ce cours et son code sont une rédaction pédagogique originale. Les sources ci-dessous servent à approfondir ; aucune promesse de reprendre leurs poids ou leur implémentation n'est faite.

- PyTorch, Learn the Basics : modules, données et boucle d'apprentissage.
- PyTorch, Automatic Differentiation : graphe et gradients automatiques.
- Ronneberger, Fischer et Brox,2015, U-Net : principe encodeur/décodeur et connexions latérales, initialement pour la segmentation.
- Goodfellow, Bengio et Courville,2016, Deep Learning : approfondissement mathématique et méthodologique, ouvrage distinct de ce cours.
- Brooks et al.,2019, Unprocessing Images for Learned Raw Denoising : importance des gains et du pipeline dans les données d'apprentissage.

La préparation B4 et ses JSON décrivent séparément les observations matérielles, les besoins de corpus et le protocole de qualification. Les14 tests du premier banc U-Net synthétique ne constituent ni une validation des rendus, ni une livraison de la bêta. Le choix d'un entraînement photographique doit attendre un contrat, des données qualifiées et des critères de sortie explicites.